OTP Stored in server or not. If not stored then how it compares

Good question — this is a very important concept in authentication systems.

🔐 Is OTP stored on the server?

Short answer:

👉 Sometimes YES, sometimes NO — depends on the method used.

✅ 1. When OTP is stored (temporary storage)

Most common method:

- Server generates OTP
- Stores it in:
  - Database OR
  - Cache (like Redis)
- Sends OTP to user (SMS / email)

🔁 Then during verification:

- User enters OTP
- Server compares:

```
entered OTP == stored OTP
```

- If matched → success

👉 Usually stored with:

- Expiry time (e.g., 2–5 minutes)
- User ID / phone number

❌ 2. When OTP is NOT stored (stateless method)

More secure and scalable approach.

Instead of storing OTP, server uses math/algorithms to verify it.

🔑 Example: TOTP (Time-based OTP)

Used in apps like:

- Google Authenticator
- Microsoft Authenticator

⚙️ How it works:

- Server and user share a secret key
- OTP is generated using:
  - Secret key
  - Current time

👉 When user enters OTP:

- Server recalculates OTP using same formula
- Then compares:

```
generated OTP == user OTP
```

✔️ No need to store OTP anywhere

🔍 Simple Comparison

Method	OTP Stored?	How Compared
Basic OTP (SMS/email)	✅ Yes	Direct match with stored OTP
TOTP (Authenticator apps)	❌ No	Recalculate using time + secret

---

💡 Key Idea

👉 If OTP is not stored, it is re-generated using the same logic and then compared.

🧠 Easy Analogy

- Stored OTP: Like checking answer from a saved paper
- Non-stored OTP: Like solving the same math again and comparing results

---

If you want, I can also explain ↪ how to implement OTP in C++ or backend step-by-step (simple version).

Explain stateless method on simple and easy way

Alright — let's make the stateless OTP method super simple 👇

🔐 What does "Stateless" mean?

👉 Stateless = Server does NOT store anything

- No OTP saved
- No database entry
- No memory used

💡 Simple Idea

Instead of saving OTP, the server uses a formula (logic) to generate it again when needed.

🔁 How Stateless OTP Works (Easy Steps)

Step 1: Secret key shared

- Server and user/app both have a secret key

Step 2: OTP is generated

- OTP is created using:
  - Secret key
  - Current time

👉 Example:

```
OTP = function(secret key + current time)
```

Step 3: User enters OTP

- User types the OTP they received/generated

Step 4: Server checks (without storing!)

- Server again runs the same formula
- Generates OTP again

👉 Then compares:

```
Generated OTP == User OTP
```

✔️ If same → Verified
❌ If different → Rejected

🧠 Real-Life Example

Apps like:

- Google Authenticator
- Microsoft Authenticator

They:

- Don't send OTP from server
- Generate OTP on your phone
- Server generates same OTP at same time

🪄 Easy Analogy

Think like this:

👩‍🏫 Teacher and student both know a secret formula

- Teacher says: "Use formula with current time"
- Student calculates → gets answer (OTP)

- Teacher also calculates → gets same answer

👉 No need to store anything!

🔥 Why use Stateless?

✔ More secure (no OTP stored)

✔ Faster (no database lookup)

✔ Scalable (handles many users easily)

⚠️ Important Note

- OTP changes every 30–60 seconds
- Based on time → called TOTP (Time-based OTP)

🧾 One-line Summary

👉 Stateless OTP = Generate → Re-generate → Compare (no storage needed)

If you want, I can also ↪ show the actual formula or simple code logic (very easy) 👍